

Retail on Shelf Object detection

Technical Report

By:

Ammar Alrousan 162141

Adnan Wadee 159788

Abdallah samarah 162691

Course: Computer Vision – Fall 2025/2026

Course instructor: Dr. Abdullah Al-amaren

Developing a Deep-Learning system for a Computer Vision Application

1 Problem statement

Design and develop a system powered by Deep Learning in order to solve a real-world problem: Object Detection within retail and supermarkets.

Various factors led to the selection of this idea. First, our team had not yet had practical experience with Object Detection, although theoretical material was covered in lectures and was found very important and influential in real-life applications, especially in smart image-based systems. Therefore, this project was deemed as a great opportunity to hone our technical skills and enhance our understanding of Computer Vision.

Our retail sector was chosen because it is among the most important sectors that is increasingly relying on Object Detection. From our analysis of the field, we identified a number of practical hurdles which still face stores—particularly in the Arab region—that lack integrated smart solutions.

Checkout congestion, from a customer's perspective, remains a major point of pain that directly deteriorates the shopping experience. As for store owners, inventory management involves a very tiring process in terms of manual audits and, of course, continuous shelf-monitoring to track stock levels. Many of these tasks are greatly labor-intensive and time-consuming besides being highly prone to human error.

With that in mind, we conceptualized an integrated intelligent system based on core engine detection. Our proposed system will include product detection on the shelf to the advanced stages: tracking in a cart, automatic billing, integrating a database for pricing, and real-time inventory analytics.

But that would involve huge technical and human resources, which are not currently within our grasp; hence, the focus of this project lies in the most important formative step: product detection in supermarket shelf images with localization through Bounding Boxes. Thus, this is in itself the basic building block for every future system feature.

The project will make use of the `SKU110Dataset_resized640`, a rescaled version of the well-known `SKU110K` dataset available on Kaggle. It has real-world images of tightly packed supermarket shelves. The system's main output is a Bounding Box for every product visible in the frame. These images are characterized by great variability of lighting conditions, camera angles, distances, and size of the products. Moreover, every image includes an extremely high

number of objects: from 170 to 230 products per frame. Thus, this task is closer to "Dense Object Detection", which is considerably more challenging than a standard detection problem.

The challenge lay in the original dataset, containing only one class for all products. Rather than consider this a limitation, we revisited this and developed a unique, intelligent pipeline that derived new classifications on the basis of the data itself. In this approach, we were able to circumvent the unrealistic expectation of manual re-labeling, which could never have been done within the time constraints of this project.

Our pipeline follows a series of strategic steps: o First, we preprocessed the data based on the available Ground Truth, extracting individual "crops" for each product. In order to ensure the quality of the data, very small segments-less than 500 pixels-were filtered out because they didn't hold enough visual information.

These crops were then embedded as Visual Embeddings using a self-supervised model. We did start by investigating the CLIP family of models, but these proved less appropriate in this context. CLIP works very strongly on the basis of the interrelation between pictures and text-aka Semantic Meaning-and, bereft of textual captions, it failed to tell apart retail products, where often subtlety and visual, rather than conceptual, differences exist.

Instead, we chose DINOv2: as a self-supervised model, it does not require text, and performs exceptionally well on fine details, such as edges, shapes, and textures. This is important in retail settings where products often appear very similar to the naked eye.

After extracting the embeddings, we performed unsupervised clustering, taking Mini-Batch K-Means to form clusters of visually similar products. To validate these clusters, we used Mosaic Visualization, which allowed us to manually review and ensure the homogeneity of each group.

This was successfully used to collapse the initial groups into 91 disjoint, non-redundant classes. This produced a multi-class dataset suitable for both Object Detection and Classification, offering a scalable foundation that can be further improved with an increase in data or even resources.

Success Criteria (The Benchmarks)

Defining Success

Our approach to success is very different from others. We don't just care about high numbers on the bottom line of our spreadsheet. We believe that a successful system should demonstrate its merit by being able to thrive under real-life conditions." In order to measure our growth effectively, we set up these tough guidelines:

- a. Overcoming the "Dense Detection" Challenge: This is because our model has to be able to perform detection even with highly dense products—so dense, for instance, that there could be more than 200 objects per image. If it cannot, it is not ready for the real-world market, which happens to be the supermarket.
- b. Precision versus Recall Balance: Here, in order to control inventory, “a false positive result equals a miss.” We are seeking a delicate balance between being sensitive and being precise.
- c. Metric Benchmarking: For tracking our metrics, we have used mAP@50 and mAP@50-95. These metrics are more than numbers as they signify the performance of the model in terms of overlapping products as well as hard IoU thresholds.
- d. Latency & Scalability: A smart system would be rendered inadequate if the time taken to process a single image takes a minute. So, we need inference times that are scalable to real-time surveillance.
- e. Generalization Performance: The model should prove that it will work on new shelf layouts with which it has never been trained. Stability on the validation set will be the only method for confirming we have not just "overfitted" the model to our particular training pictures.
- f. Basis for Downstream Processes: Ultimately, these bounding boxes must be accurate enough to be used directly in the subsequent stages, such as automatic billing or immediate notifications of out-of-stock products.

2. Research on Neural Networks and Architectures

2.1. Foundations for a Retail Detection Neural Network

Retail Object Detection is not a typical computer vision task, it ranks among the more complex ones. What is necessary is that it be performed by a system capable of correctly examining the local details of a given image and, at the same time, having a global vision of the image. The basic task is the accurate identification of the positions of hundreds of densely packed objects.

In fact, in all retail settings, we must necessarily contend with various so-called “environmental headaches,” including:

- **Extreme object density:** High object/pixel ratios
- **Visual Mimicry:** Products tend to be very similar in terms of appearance.
- **Scale Variance:** The extreme variation in scale from a pack of gum to a bottle of detergent.
- **Occlusion & Lighting:** There is partial occlusion among items, hiding some in the shadows, due to harsh lighting.

The standard image processing fails in this scenario. As a result, Deep Neural Networks (DNNs) emerge as the only feasible way ahead. The current state-of-the-art architectures utilize the following custom neural layers to mitigate this problem:

Convolutional Neural Networks (CNNs): CNNs are still considered the "workhorse" of vision. These models primarily serve as the Backbone for identifying low-level feature maps: edge detection, texture, and basic shapes. This step of the process essentially gives the system the raw visual information it needs.

Fully Convolutional Networks (FCNs): FCNs eliminate the need for Fully Connected layers. This enables the network to retain its geometric structure. This is a crucial point for localization. Localization is more about where a product is, rather than which product is there. FCNs provide the basic architecture for single-stage detectors.

Region-based Architectures (Two-Stage): Region-based architectures divide the task into two: the first, suggesting “interesting” regions, followed by classification of the regions. Though famous for being highly accurate, particularly at detecting small, overlapping objects, they are computationally intensive and therefore slow in practical, high-speed retail applications.

One-stage Detectors: To emphasize speed, these models forgo the proposal stage and make direct estimates in one pass. For intelligent retail applications demanding nearly instantaneous responses, the efficiency offered by one-stage detectors may prove more valuable than their tiny losses in accuracy.

Transformer-based Models & Self-Attention: Transformers revolutionized this by introducing the concept of Self-Attention to vision. While CNNs take small patches, Transformers view the big picture and grasp how objects are related to each other in the full frame. This global context becomes a lifesaver in cluttered shelf scenes.

Hybrid Architectures: CNN + Transformer form the current sweet spot of research. These architectures blend the spatial efficiency of a CNN with the relational intelligence

of the Transformer. For such a complex retail environment, these hybrids will offer a nice balance of speed and precision, hence becoming ideal for the high-density requirements of our projects.

2.2 Modern Architectures in Object Detection

Based on the neural foundations established in the previous section, the current section will analyze the architectures which have established state-of-the-art approaches in the retail setting. Such architectures close the gap between theory and industry-adoptable solutions.

YOLO: Arguably the most recognizable name when it comes to one-stage detectors, YOLO transformed the industry by invoking the idea of detection as a singular regression task. By looking at the image only once, it puts out bounding boxes and predictions at the same time. In the retail world, YOLO's key "selling" property has always been its sheer speed. For a system required to analyze the description of hundreds of articles on the shelf within the timescale of milliseconds, YOLO has always been one of the best contenders even when it came to extremely dense objects.

SSD (Single Shot Detector): The SSD follows a one-stage philosophy similar to YOLO. The SSD brings in a new concept called multi-scale. This allows them to make use of various layers of the convolutional neural network to detect objects of various sizes. In retail applications, the scenario involves identifying objects of varying sizes at the same time. Examples of such objects would be candies and detergents. The SSD can identify these with higher localization.

Faster R-CNN: As the benchmark for two-stage detectors, the Faster R-CNN detector values detection precision over every other parameter. The detector utilises the Region Proposal Network (RPN), which scouts for target objects for the purpose of classification. Although highly effective for detecting small, overlapped products that are a challenge in the retail industry, the detector's multi-step detection process suffers from an impediment in the form of an ensuing lag.

DETR (Detection Transformer): Being a paradigm shift, the DETR transformer abandoned the need for the "anchors" and "non-maximum suppression" techniques used in traditional models. By incorporating the use of the self-attention technique, it recognizes objects within the image and, therefore, the entire context behind a shelf. The downside of this level of accuracy, however, lies in the computational requirements for the training process.

RT-DETR (Real-Time Detection Transformer): The weaknesses of the original DETR motivated the creation of RT-DETR to introduce the accuracy of transformers to the world of real-time detection. RT-DETR can be thought of as a "hybridization" of its architecture with a CNN as a feature extractor and a transformer to perform relational

reasoning. In our case, RT-DETR offers us the accuracy required in a dense retail setting but with the speed of its predecessors.

2.3 Comparative Analysis of Modern Architectures

After highlighting the types of models that can be utilized, it becomes necessary to compare these different architectures. We will compare these architectures based on their structure, major components, as well as the natural trade-offs between the strengths and weaknesses of each model. These comparisons will help us explain why we chose a particular technical approach for this particular project.

Architecture	Design Strategy & Components	Key Advantages	Major Drawbacks
YOLOv8	A fully convolutional , one-stage detector. It utilizes a CNN Backbone for feature extraction, a Neck for multi-scale feature fusion, and a Head for direct bounding box and class prediction.	Blazing fast inference speeds; a relatively simple and deployable structure ideal for high-density retail scenes.	Can suffer from lower relative accuracy in cases of extreme overlap or when dealing with ultra-small objects compared to two-stage methods.
SSD	A one-stage framework built on a CNN Backbone that generates additional convolutional feature maps at multiple scales to handle varied object sizes.	Transparent architecture with a respectable balance between speed and precision; well-suited for varying product dimensions.	Performance tends to degrade in highly congested scenes compared to newer architectures like YOLOv8.
Faster R-CNN	A two-stage architecture featuring a CNN Backbone and a Region Proposal Network (RPN) to scout for objects, followed by a dedicated classification head.	Exceptional accuracy, particularly when localizing small and overlapping items in complex layouts.	High computational latency and architectural complexity make it unsuitable for real-time retail applications.
DETR	An end-to-end Transformer-based model using a CNN Backbone followed by a Transformer Encoder-Decoder with self-attention to grasp global image relationships without anchors or RPNs.	Provides a cleaner, more conceptually holistic understanding of the entire scene.	Notoriously high computational cost; slower training and inference cycles make it less efficient in certain practical scenarios.
RT-DETR	A hybrid model utilizing a CNN Backbone for feature extraction and an optimized Transformer Decoder to process global context with lower latency than standard DETR.	Strikes a sophisticated balance between accuracy and speed; excels in complex, high-overlap environments.	Structurally more complex than traditional one-stage models and requires more substantial hardware resources.

Table 1: Modern architectures used to solve the problem

Justification of Architecture Selection

As for our motive for choosing **YOLOv8** and **RT-DETR**, it was due to our intention of investigating two different, albeit complementary, approaches in the realm of object detection:

- **YOLOv8**: We choose this as our main one-stage model. The fully convolutional architecture of this model, in addition to having the optimized pipeline, is considered to be the "gold standard" in terms of deployability when speed is an essential requirement for processing several items per frame.
- **RT-DETR**: To further extent the limits of excellence in accuracy, we introduced RT-DETR. It bridges the CNN and the Transformer and brings in a further "contextual" interpretation of the shelf that is very relevant in the retail domain because shelf simulations and packaging stacking might mislead trivial AI models.

The Strategy: By leveraging both models, we're not simply trying to determine which one is the "winner." Rather, we're building a powerful framework which leverages the best of both worlds in terms of both raw velocity and contextual accuracy. This is what will allow us to pit a classic, high-speed solution against this new, contextual solution and tailor our solution specifically to this retail-centric task.

3. Models' development and training

3.1 Dataset

3.1.1 Dataset Source and Domain

The dataset used is **SKU110 dataset_resized640**, The SKU-110k dataset is widely used for training and evaluating deep learning models in object detection tasks, especially in densely packed scenes such as **retail shelf displays**, this dataset contains the labels comprising the bounding boxes (only one class for all detections), as well as all the resized images for the SKU110K dataset. This is resized to benefit the training by removing the pre-processing step .& avoiding the overhead of resizing the images during training[1].

3.1.2 Data Collection and Annotation

Key Features

- SKU-110k contains images of store shelves from around the world, featuring densely packed objects that pose challenges for state-of-the-art object detectors.
- The dataset includes over 110,000 unique SKU categories, providing a diverse range of object appearances, Although the original dataset contains a large number of SKU categories, all objects were treated as a single class in this study.
- Annotations include bounding boxes for objects and SKU category labels.

3.1.3 Dataset Structure

The SKU-110k dataset is organized into three main subsets:

- Training set: This subset contains 8,219 images and annotations used for training object detection models.
- Validation set: This subset consists of 588 images and annotations used for model validation during training.
- Test set: This subset includes 2,936 images designed for the final evaluation of trained object detection models.

3.1.4 Image Characteristics

The images in the SKU110k (Resized 640) dataset are real-world retail shelf images with a high object density per image, where objects tend to be arranged closely together with small sizes, making object detection even harder. The images contain real-world scenarios characterized by high object density per image.

All the images used in the version of the dataset used here are of uniform size, which is 640x640 pixels, to allow the deep learning models to train with the same size of input.

There is variation in the amount of lighting, background clutter, and camera views of the objects. In addition, a considerable amount of occlusion of the objects by each other also affects the dataset. Bounding box statistics show that the class instances tend to consist of small image areas, which increases the difficulty of detecting fine-grained instances. From these statistics, it can be observed that the dataset is well-suited for comparing the performance of object detection models because of its characteristics.

3.1.5 Dataset Preparation and Preprocessing

Preliminary procedures were performed to process the incoming data and to produce data sets of higher quality (some procedures have been listed below specifically) and increased reliability.

The images that were to be included in the data set were resized to 640 x 640 pixels prior to training the models. This allowed us to train the models using far less computational power than would have been necessary if we had to resize the images during model training.

The next step was to verify the data set's integrity by confirming that each image contained an accurate corresponding annotation file. Verification was conducted on all images to ensure that none of the images contained corrupted files. The annotation files were also checked to ensure that all individual bounding boxes were formatted correctly and to count the number of varying-sized objects that were present in the annotated data set.

Finally, we established a method for organizing our data sets, with each data set following a structured pattern that had been designed specifically to work with the object detection algorithms used in this study.

The combination of all of the steps listed here has resulted in the highest possible level of consistency for all of our data sets, and therefore, a high degree of stability during the training of the models and an ability to provide a high level of quality in experimental test results produced by these models.

3.2 Training and validation

The training phase of the models in this phase we used two object detection architectures **YOLOv8m** and **RT-DETR**, and for each architecture we used several hyperparameter configurations, we chose 3 different combinations of parameters to choose best one depending on Validation set performance and choose best values for best performance based on in mAP50-95 standard (standard metric on object detection applications).

3.2.1 Architectures

The YOLOv8m is a powerful object detection system. It can accurately and efficiently detect a variety of objects and correctly classify their location in a single pass through the network. The network has been designed specifically for high-density object scenes, which include many objects placed close together.[2]

RT-DETR: The RT-DETR is an object detection model that is based on an attention-mechanism architecture (also sometimes referred to as a transformer). The RT-DETR uses the attention mechanism to allow many-to-many relationships to be modelled. This allows it to achieve nearly the same level of accuracy compared to traditional DETR models, but with a lower average inference time.[3]

3.2.2 data split

The data is divided using the threefold split method, where the data is allocated **70% for the training dataset**, **5% for the validation** dataset, and the remaining **25% for the test** dataset.

The major part of the data was used for the training set to allow the models to efficiently learn from the complexity of the data, especially considering the fact that the detection of

objects in the retail shelf context requires a considerable number of samples owing to the object density and size.

A proportionally smaller number of data points, to the extent of 5%, was allocated to the validation set, simply because the end use of the validation set is only to determine the performance of the model and the corresponding hyper parameters during the training process, and not training per se. This was sufficient due to the larger volume of data available to prevent over-learning.

On the other hand, a comparatively large proportion of 25% was allocated to the test set in order to obtain a robust and reliable final assessment of the performance of the model on non-visible data, which is of extreme importance in practical applications of object detection.

For a fair comparison of both models, the same split was used for both architectures (YOLOv8m and RT-DETR).

3.2.3 Training Procedure

At the beginning of the training process, the models were initialized using pretrained weights, while the dataset configuration was defined through a YAML file.

The number of epochs to train for in the hyperparameter tuning experiments and in the final train stage was set to 10 epochs and 25 epochs, respectively, in order to balance convergence speed during tuning and model stability during final training.

All the experiments were performed on the images with the input resolution fixed at 640×640 pixels, following the resized set.

The training environment used is the Kaggle environment, which supports two GPUs of 15 GB VRAM each, and the hyperparameter experiments and the training process both employed multi-GPU training. During the training phase, the validation process was conducted per epoch to check the performance of the model on the validation set data and help to determine the best hyperparameter set.

3.2.4 Hyperparameters Considered

In targeted hyperparameter tuning, each architecture was explored by varying a small set of important hyperparameters. For example, learning rate, batch size, and optimizer were analyzed for optimal validation performance. The hyperparameters examined, along with their respective values, are shown below in Table 2.

Hyperparameter	Description	Architecture 1 (YOLOv8m) – Value(s)	Architecture 2 (RT-DETR) – Value(s)
Learning Rate lr0	Controls the step size used to update the model weights during training. Smaller values provide more stable convergence, while larger values accelerate training but may cause instability.	1e-3, 5e-4	1e-4, 5e-4
Batch Size	Defines the number of training samples processed simultaneously before updating the model weights. Larger batch sizes improve computational efficiency, while smaller batch sizes may improve generalization.	8, 16	8, 16
Optimizer	Specifies the optimization algorithm used to update the model parameters during training. Different optimizers affect convergence speed and stability.	AdamW, SGD	AdamW, SGD
Image size	Indicates the spatial resolution of the input images fed to the model. A fixed image size ensures consistent input dimensions during training.	640 × 640	640 × 640
Epochs	Represents the number of complete passes over the training dataset. Different values were used for tuning experiments and final training.	10 (tuning), 25 (final)	10 (tuning), 25 (final)

Table 2: Description of the hyperparameters considered for each architecture

3.2.5 Hyperparameter Combinations

For each architecture we tried 3 combinations each one different from other with at least one parameter, all experiments with 10 epochs.

Architecture	Combination of hyperparameter values	Training performance (Final loss)	Validation performance(mAP50-95)
YOLOv8m	(LR =1e-3 , Batch = 8, Optimizer =AdamW)	2.879	0.5957
	(LR =5e-4, Batch = 8, Optimizer =AdamW)	2.870	0.5973
	(LR =1e-3 , Batch = 16, Optimizer =SGD)	2.955	0.5827

RT-DETR	(LR = 1e-4, Batch = 8, Optimizer =AdamW)	1.135	0.526
	(LR =5e-4 , Batch = 8, Optimizer =AdamW)	1.041	0.553
	(LR = 1e-4, Batch = 8, Optimizer =SGD)	1.896	0.286

Table 3: Combinations of the hyperparameter values and corresponding performance achieved

3.2.6 Model Selection Criteria

We chose the best models based on the highest validation performance using mAP50-95 metric.

For YOLOv8m, The second configuration achieved the best validation performance while maintaining a lower training loss, indicating better convergence stability compared to the other tested combinations.

And for RT-DETR, The second configuration achieved the best validation performance while maintaining a lower training loss, indicating more stable convergence. Increasing the batch size and switching to SGD resulted in slower convergence and significantly lower detection performance.

3.2.7 Learning Curves

We extracted the learning curves for the best combinations for each architecture:

YOLOv8m :

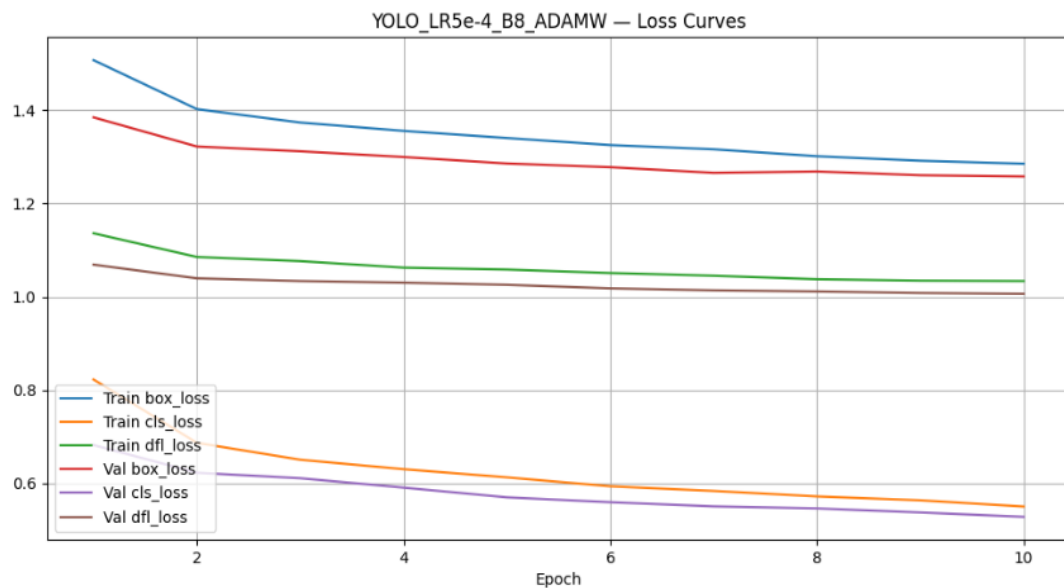


Figure 1: Training and validation loss vs epochs

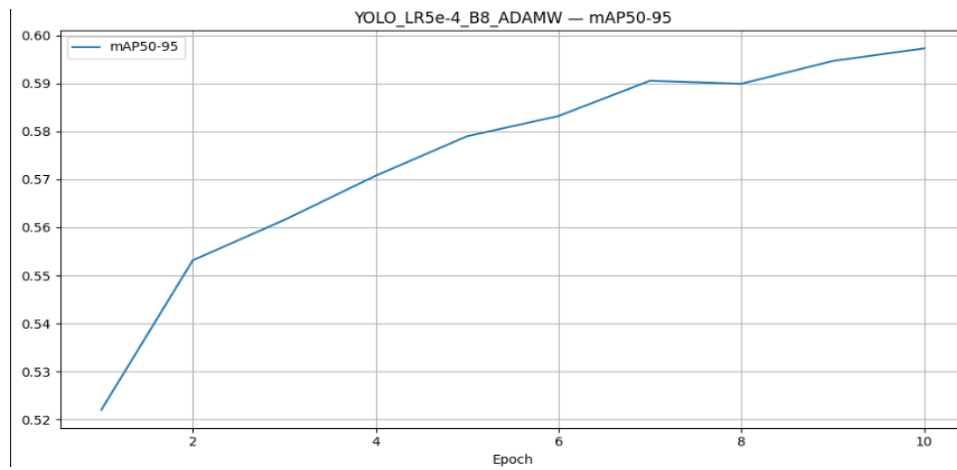


Figure 2: Validation mAP50-95 vs epochs

RT-DETR:

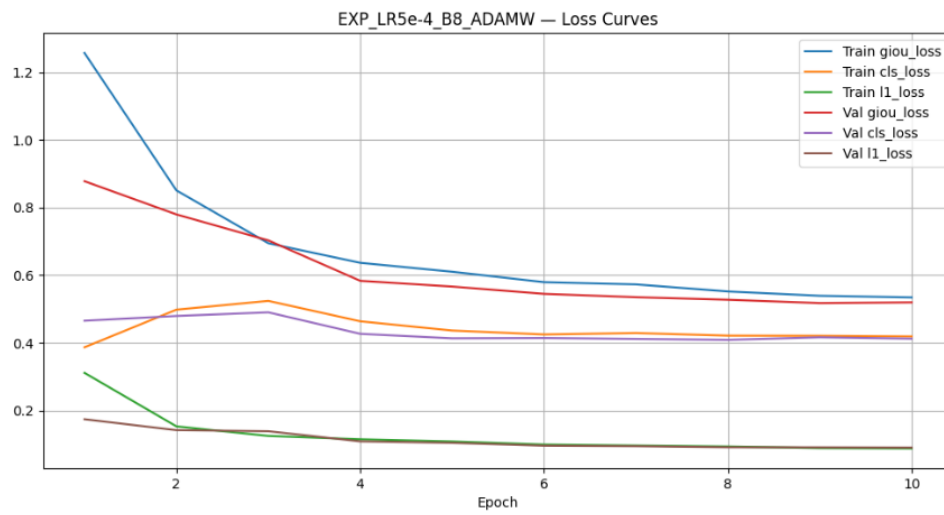


Figure 3: Training and validation loss vs epochs

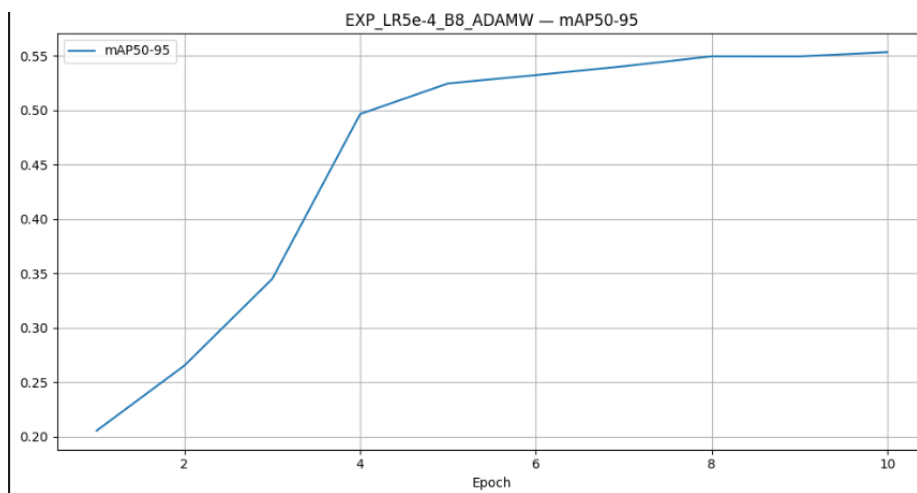


Figure 4: Validation mAP50-95 vs epochs

3.2.8 Final Training Configuration

The last phase of training utilized the same set of hyperparameters as those determined during the hyperparameter optimization trials. The learning rate, the batch size, and the optimizer were set at the configuration from the hyperparameter optimization trials, which yielded the best performing Validation setup. As a result, the image input size was established to be 640×640 px to approximate the size of the original dataset. The last training phase lasted for 25 Epochs to allow the study to obtain enough time for the interaction between the data and the model in order to achieve a stable model convergence. During the final training phase, the three-way data split strategy of Training, Validation, and Test remains constant to each of the three prior phases.

Training Setting

The Kaggle Cloud environment was used for Training all experimental studies. Two Graphical Processing Units (GPUs) were utilized to improve computation time and reduce overall Training time. Throughout the course of these studies, GPU acceleration was enabled, and model validation occurred during the Training studies to allow for continuous monitoring of the model's performance and process of convergence.

Transition to Evaluation Stage

The model developed from the last training stage has been saved and is now ready for evaluation. Specifically, the test dataset will be utilized to examine the degree to which the current model generalizes, as well as to provide a fair comparison of the current model with other architectures.

4. Testing Previous Developments

4.1 Testing

4.1.1 Testing

After completing the training and parameterization phases, a standardized and fair protocol was applied to select the best models from each architecture (YOLOv8 and RT-DETR). The evaluation was performed on the SKU-110K test dataset, which had not undergone training or validation, and an objective assessment of the model's generalizability was conducted.

For each architecture, the following steps were implemented:

From each base, the best model was selected:

For each architecture, several models were trained on different hyperparameter values, such as learning rate, batch size, and optimizer type. The best model with the highest mAP quality (0.5:0.95) was selected for validation, as this metric reflects both detection accuracy and positioning quality. This integrated measurement is crucial for diagnostics and positioning in congested component environments.

Evaluation on the Test Set:

The final models were evaluated on the test set, and the evaluation results were reported using, mAP, Precision, Recall, and F1-score.

Visual and Analytical Evaluation:

After conducting numerical metrics, we also performed a visual evaluation by displaying the detection results on real images of shelves. We analyzed the accuracy, recall, and confidence index (F1) parameters, along with confusion matrices, to understand the model's behavior in crowded and congested environments.

4.1.2 Evaluation Criteria:

the following evaluation criteria is used to analyze the effectiveness of the proposed system in accurately detecting high-quality products:

Mean Accuracy (mAP):

The mAP metric was used for the basic evaluation process to detect items. An mAP value of 0.5 million is considered good if the model is able to correctly detect the product.

mAP @ 0.5:0.95 measures the accuracy of positioning against the stricter IoU threshold index and is important for inventory management, such as shelf monitoring and stock management.

The best-performing model from each architecture was evaluated on the test set, and the results are summarized in **Table 4**.

Architecture	Best combination of hyper-parameter values	Values obtained for the evaluation metrics
YOLOv8m	Learning rate = 0.0005, Batch size = 8, Optimizer = AdamW	mAP@0.5 = 0.9359 , mAP@0.5:0.95 = 0.6185 , Precision = 0.9079 , Recall = 0.8828 , F1-score = 0.8952
RT-DETR-L	Learning rate = 0.0005, Batch size = 8, Optimizer = AdamW	mAP@0.5 = 0.9029 , mAP@0.5:0.95 = 0.5933 , Precision = 0.8793 , Recall = 0.8702 , F1-score = 0.8748

Table 4: Values for the evaluation metrics achieved on the test set for the best model from each architecture

4.1.3 Test Results Discussion:

Table 4's findings demonstrate that both architectures performed well on the test set. Better localisation accuracy and fewer false detections are indicated by YOLOv8's somewhat higher mAP and Precision ratings. However, RT-DETR showed competitive Recall and F1-score, demonstrating a good capacity to recognise the majority of products even in situations with chaotic and thick shelves.

Both models are successful for dense retail product recognition, according to the confusion matrices and qualitative visualisations. RT-DETR shown resilience in managing intricate spatial interactions between closely spaced objects, whereas YOLOv8 was more efficient in inference speed.

4.2 Models Analysis

4.2.1 YOLOv8 Model Analysis

Comparing Training and Validation Losses:

The training results for YOLOv8 showed that the Training vs. Validation Losses were very close at the end of the training process. Here's how small the difference was:

Train Box Loss ≈ 1.2548

Validation Box Loss ≈ 1.2403

Gap ≈ -0.0146

This small difference clearly indicates that the model did not over-remember the training data, but rather learned a general representation capable of generalization.

Performance Comparison Between Validation and Test:

When comparing performance metrics for validation and testing, we observe that the values are similar. The model achieved:

On Validation:

mAP@0.5 ≈ 0.923

mAP@0.5:0.95 ≈ 0.609

On Test:

mAP@0.5 $:: 0.9359$

mAP@0.5:0.95 $:: 0.6185$

This slight improvement on testing indicates that our model is well-balanced and does not suffer from overfitting, nor does it underfit.

Final Conclusion for the YOLOv8 Model:

Based on the similarity of loss and consistent performance across different groups, it can be said that the YOLOv8 model:

Does not suffer from overfitting or underfitting.

Achieves a good balance between learning and generalization.

4.2.2 RT-DETR Model Analysis

Learning Curve Behavior:

The RT-DETR model's learning curves showed a gradual improvement in Precision, Recall, and mAP values as the training epochs progressed, without any sharp fluctuations or sudden drops in the validation set's performance. This is a positive indicator of the training process's stability.

Validation and Test Comparison:

When comparing the validation and test results of the final model, we found the values to be very similar:

Validation Results:

mAP@0.5 $\approx$ 0.889

mAP@0.5:0.95 $\approx$ 0.584

Test Results:

mAP@0.5 : 0.9029

mAP@0.5:0.95 : 0.5933

We observed no decrease when moving from Validation to Test. This indicates that the model did not over-retain or over-retain the training data.

Conclusion for the RT-DETR Model

The final results indicate that the RT-DETR model:

Does not show any signs of overfitting and is not underfitting.

Demonstrates good generalizability in high-density shelving images.

4.3.1 Detection Accuracy and Localization Quality:

Explanation: Detection accuracy and location quality are among the most important factors in this project because the shelf images contain a large number of products that are very close together and some overlap. Therefore, the mAP@0.5:0.95 metric is a key indicator of the model's ability to accurately locate each product with sufficient precision for inventory management. The results shown are as follows:

YOLOv8:

mAP@0.5 = 0.9359

mAP@0.5:0.95 = 0.6185

RT-DETR:

mAP@0.5 = 0.9029

mAP@0.5:0.95 = 0.5933

Both architectures achieved excellent results in the product detection task on the shelves, but the results show that YOLOv8 excels in location accuracy. This indicates... Higher and stronger ability to separate stuck products with greater accuracy than RT-DETR.

4.3.2 (Precision and Recall)

In shelf-tracking projects and applications, accuracy and recall represent two different types of errors. Higher accuracy means fewer false detections, while higher recall means fewer missing products. A scientific system requires a balance between these two.

These are the results:

YOLOv8

Precision: 0.9079

Recall: 0.8828

RT-DETR

Precision: 0.8793

Recall: 0.8702

These values indicate that YOLOv8 slightly outperforms the previous system by producing fewer false detections while maintaining a high product detection rate. This achieves a crucial balance in a crowded environment that can lead to false or missing detections and poor inventory management decisions.

4.3.3 (F1-score):

F1-score Why do we use it ? To assess the balance between accuracy and recall, especially in crowded and high-density shelving environments. Each type of error has a direct impact on the system, and in these types of applications, sacrificing accuracy is difficult because it leads to an increase in error predictions. Similarly, sacrificing recall is unacceptable because missing products leads to incorrect decisions, such as assuming a product is not on the shelf.

The F1-score represents a comprehensive indicator of performance quality because it combines precision and recall into a single value, making it suitable for evaluating the overall stability of the model in complex, real-world environments.

The results showed:

YOLOv8 showed an F1 of 0.8952

RT-DETR showed an F1 of 0.8748

These values indicate that both models achieve an excellent balance between reducing error and increasing detection accuracy. However, the results showed that YOLOv8 performed better, demonstrating more stable performance in scenarios with a larger number of closely spaced products. This means that YOLOv8 can maintain high accuracy without missing a significant number of products, making it more suitable for use in shelf monitoring systems and inventory management processes.

4.3.4 Inference Speed:

Inference speed is a critical factor in evaluating model effectiveness, especially in in-store systems where a large number of images may be analyzed in a short time. A practical system needs to be able to process images quickly while maintaining a high level of accuracy to ensure rapid response and continuous shelf status updates.

The inference time for each model was measured using a single image from the test set. This time includes the time required to process the image before input, execute the model, and process the output.

Results:

YOLOv8:

Inference time $\approx$ 38 ms per image

RT-DETR:

Inference time $\approx$ 70 ms per image

The results show that YOLOv8 is approximately twice as fast as RT-DETR, a significant difference for near-instantaneous applications. This speed advantage makes YOLOv8 more suitable for systems operating continuously in stores, as it can process a larger number of images in the same time period without compromising detection accuracy. In contrast, although RT-DETR performs well in terms of accuracy, its higher execution time may limit its use in scenarios that require a rapid response.

4.4 Effectiveness Assessment

This section compares YOLOv8, RT-DETR in terms of object detection for accuracy, memory utilization, training and inference costs as well as their applicability to retail scene with high product similarity and density.

4.4.1 Memory Requirements

Memory cost was measured by two metrics: total number of trainable parameters and the size of final model after training. These features are critical in consumer applications, where models must run every time for long and storage-compressed periods.

The resulting trained YOLOv8 model file was about 50 MB in size and had approximately 25 million parameters. The RT-DETR model was much larger, with a file size between 63 and 65 MB and approximately around 63 million parameters. This increased size is due to the inherent complexity of switch-based architectures that depend on multi-headed self-attention mechanisms and finer feature representations.

At the same time, the smaller memory requirements of the YOLOv8 model facilitated greater flexibility in using, updating or saving multiple models simultaneously. The increased size of the RT-DETR model hinders its deployment in resource-constrained or peripheral retail surveillance environments, even though its memory footprint remains affordable provided a GPU-based computing platform is used.

4.4.2 Requirements for Training Computing

By examining convergence patterns and architecture complexity, the training computing efficiency was assessed. Both models were trained using free Tesla T4 or P100 GPUs with 16 GB of VRAM from the Kaggle platform, utilising the same SKU-110K dataset components.

The YOLOv8 model achieved stable performance over fewer training cycles and showed faster convergence during training. Its single-stage convolutional design, which facilitates effective feature extraction and enhances overall performance, is consistent with this behaviour.

To attain comparable performance, the RT-DETR model needed much longer training times. Convergence was slowed by the use of switch-based attention mechanisms, which raised the computational requirements per training cycle. Even though the training process was stable, the increased complexity led to higher computational costs, especially when multiple trials or frequent retraining were needed.

In actuality, this makes the YOLOv8 model more appropriate for software development lines that need to update their models on a regular basis to accommodate changes in seasonal inventory or new product designs.

4.4.3 Efficiency of Inference

When assessing efficacy, inference speed is crucial, particularly in retail shelf monitoring systems that must process a large number of images nearly instantly. Individual photos from the test set were used to assess inference time under identical circumstances.

The RT-DETR algorithm took about 70 milliseconds per image, whereas the YOLOv8 algorithm achieved an average inference time of about 38 milliseconds per image. This suggests that in terms of inference, YOLOv8 was almost twice as quick. This speed difference led to higher throughput and quicker shelf status and inventory updates in situations requiring ongoing monitoring.

RT-DETR's slower inference speed limited its usefulness in time-sensitive scenarios, like real-time shelf availability monitoring or quick identification of misplaced products, even though it was still appropriate for offline or batch processing applications.

4.4.4 Real-World Use

Effectiveness in real-world retail settings depends on striking a balance between operational constraints, computational efficiency, and detection performance. Images of superstore shelves frequently show hundreds of closely spaced items with frequent obstructions, necessitating models that can provide both efficiency and long-term dependability.

The YOLOv8 model's small size, few parameters, quick training convergence, and quick inference speed make it ideal for central servers handling enormous volumes of images and retail systems with limited resources. Continuous operation at low computational costs is made possible by its efficiency.

The RT-DETR model showed competitive detection performance and the capacity to represent intricate spatial relationships among closely related products.

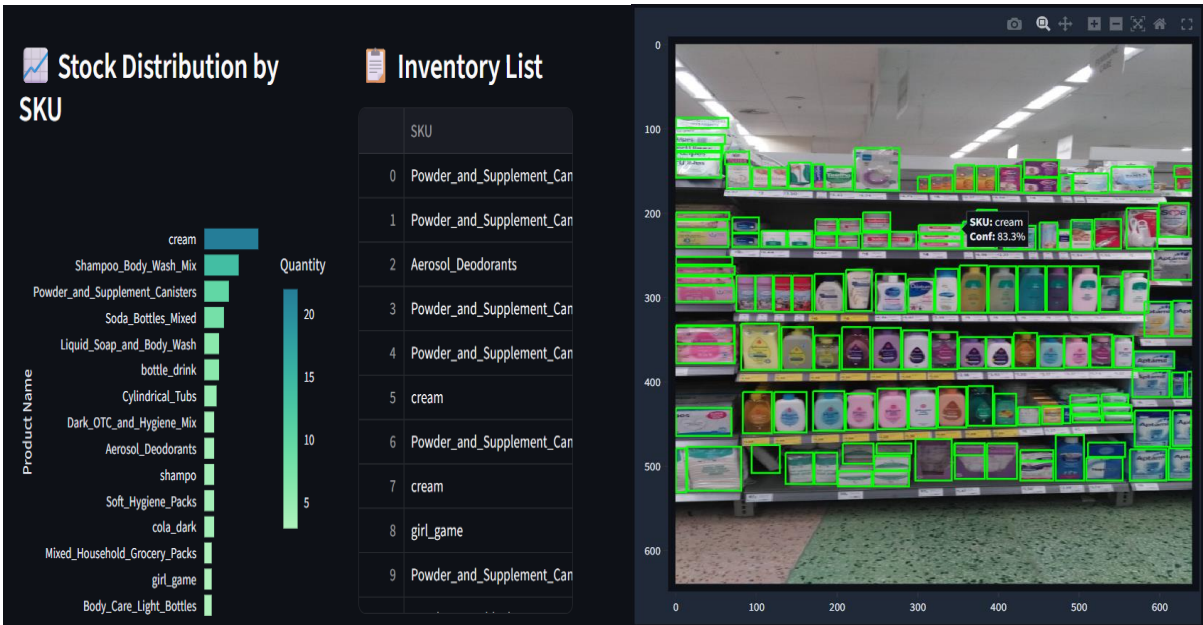
Classification Experiment Using ResNet-50

As an additional (non-core) project task, an image classification experiment was conducted using a ResNet-50 model pre-trained on ImageNet. This experiment was performed on the same project data to explore the possibility of adding a classification layer to differentiate between products/categories as a complementary step to the computer vision path.

The model's classifier head was modified to match the number of categories in our data, and then fine-tuning was performed on the training data using optimization procedures such as AdamW and ReduceLROnPlateau, along with class weights to address data imbalances.

However, due to limited computing resources (limited training time/GPU and memory constraints), we were unable to complete the experiments as intended to achieve stable and reliable results. These limitations were communicated to the project manager during implementation. Therefore, this experiment is presented as an additional/exploratory attempt only and is not part of the core system delivery.

To illustrate this, an example of an input image before classification, and an image of the initial results that did not reach the expected form, will be attached, with the aim of documenting the experiment and its initial results.



Model Selection and Configuration

The interface provides a control panel that allows the user to select the detection engine, where the user can switch between YOLO and RT-DETR models. This enables direct comparison between different architectures using the same input image. In addition, a sensitivity threshold slider is included, allowing the user to control the confidence level required for displaying detections. This feature helps balance between detecting more objects and reducing false positives, which is important in dense retail shelf environments.

Input Source

The system supports image-based input through an image upload option. The user can upload a JPG or PNG image of a retail shelf using the upload panel. Once an image is uploaded, it is displayed in the interface under the *Source Feed* section, allowing the user to verify the input before running inference.

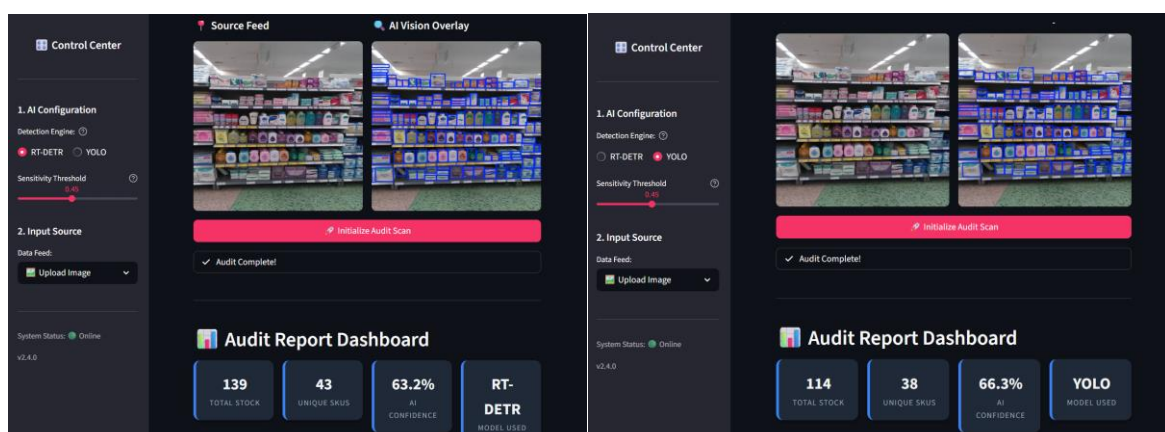
Output Visualization

After initializing the detection process, the system displays the prediction results in a dedicated output window. Since this project focuses on object detection, the output is presented as:

- The original input image overlaid with bounding boxes around detected products.
- Each bounding box corresponds to a detected product instance, optionally including confidence scores.

The interface shows the original image and the processed image side by side.

Example of the interface:



Real-Time Usability

One of the primary end-user requirements in retail environments is the ability to process shelf images efficiently and with minimal delay. Based on the conducted experiments, the final YOLOv8 model achieved an average inference time of approximately 38 ms per image, excluding preprocessing and postprocessing. This performance is sufficient for near-real-time applications such as periodic shelf scanning, automated inventory updates, and detection of missing or misplaced products.

In contrast, RT-DETR required significantly longer inference time (approximately 70 ms per image), which limits its suitability for real-time or high-throughput scenarios. While RT-DETR remains usable for offline analysis, its latency makes it less practical for deployment in systems that require fast response times.

Memory and Deployment Constraints

From a deployment perspective, memory efficiency is critical, especially when models are expected to run continuously or be deployed on resource-limited systems. The final YOLOv8 model contained approximately 25 million parameters and resulted in a saved model size of around 50 MB, making it relatively lightweight and flexible for deployment.

RT-DETR, on the other hand, contained more than 60 million parameters and produced a noticeably larger model size. Although this remains acceptable on GPU-equipped servers, it reduces flexibility in edge-based or multi-model deployment scenarios. Therefore, for end users such as retail chains that require scalable and cost-efficient systems, YOLOv8 better satisfies memory-related constraints.

Prediction Performance in Dense Shelf Environments

The effectiveness of the solution was primarily evaluated in dense shelf scenarios, where images often contain hundreds of tightly packed products. The final YOLOv8 model achieved a mAP@0.5 of 0.9359 and a mAP@0.5:0.95 of 0.6185 on the test set, indicating strong detection performance and accurate localization.

Additionally, the model demonstrated a good balance between Precision (0.9079) and Recall (0.8828), resulting in an F1-score of 0.8952. This balance is essential for inventory monitoring systems, as both false detections and missed products can lead to incorrect stock analysis. These results confirm that the implemented solution meets the core accuracy requirements of the end user.

However, visual inspection of prediction outputs showed that extreme shelf density and heavy occlusion can still lead to overlapping bounding boxes or occasional missed detections. This indicates that while the system is effective, it is not yet perfect in the most challenging scenarios.

Limitations of the Current Solution

Despite its strong performance, the implemented solution has several limitations. First, the system operates on static images and does not leverage temporal consistency that could be

obtained from video streams. Second, the two-stage pipeline (detection followed by classification via cropping) makes the overall system sensitive to errors in the detection stage. Any missed detection directly affects the classification outcome.

Furthermore, the model was trained on images resized to a fixed resolution (640×640), which may limit performance when dealing with extremely small products or high-resolution shelf images.

4.6.3 Proposed Future

On the basis of the above discussion and analysis, a number of possible areas of improvement can be identified for the future:

- Gathering appropriate and accurate data for better classification
- Developing the system to include the remaining stages mentioned in the problem (customer tracking, handling all different cases of product location and form, calculating the cost of the basket for the customer and linking it to the company's database, managing warehouses and inventory automatically).
- We completed the modifications to the models when resources were provided, and two improved models were created and modified by us. Our aim was to increase the results and reduce the parameters.
- Supporting Multi-Frame Processing (Video-based Analysis)
- Incorporation of temporal information from the video may enhance the stability and minimize product loss for crowded scenarios.

Implementation of a Unified (End-to-End) Classification System Rather than dividing detection and classification into completely separate processes, there could be an integrated model designed that incorporates the two steps. Increasing Data Range

Introducing Including data with differing lighting conditions, angles of photography, as well as new products, can increase the ability to generalize in practical settings.

Reference

- [1] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-End Object Detection with Transformers,” in Proc. European Conf. Computer Vision (ECCV), 2020, pp. 213–229.
- [2] Y. Lv, S. Zhang, Y. Wang, J. Qi, and X. Xu, “RT-DETR: Real-Time Detection Transformer,” arXiv preprint arXiv:2304.08069, 2023.
- [3] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [4] X. Goldman, A. Herzig, and L. Zelnik-Manor, “Dense Object Detection in Retail Scenes,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2019.
- [5] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL Visual Object Classes (VOC) Challenge,” Int. J. Comput. Vis., vol. 88, no. 2, pp. 303–338, 2010.
- [6] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in Proc. European Conf. Computer Vision (ECCV), 2014, pp. 740–755.
- [7] J. Redmon and A. Farhadi, “YOLO9000: Better, Faster, Stronger,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2017.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016.